

多文件编程

复习

- | | |
|---------------|----|
| 1. 概念 | 了解 |
| 2. typedef 声明 | 了解 |
| 3. 局部变量 | 重点 |
| 4. 全局变量 | 重点 |
| 5. 静态变量 | 熟悉 |
| 6. C++存储类 | 熟悉 |

目录

- | | |
|---|----|
| 1. 为什么要多文件编程 | 了解 |
| 2. 多文件编程具体步骤 | 重点 |
| 3. <code>include <></code> 和 <code>#include ""</code> 的区别 | 熟悉 |
| 4. 防止头文件重复包含 | 重点 |

什么是多文件编程

为了**方便分工或后期**的维护，分散代码应遵循一个基本原则：实现相同功能的代码应存储在一个文件中。

C++ 代码文件根据后缀名的不同，大致可以分为如下几类：

.h：头文件

.hpp：头文件，header plus plus 的缩写，混杂着 .h 的声明 .cpp 的定义，OpenCV 采用

.cpp：源文件，windows

.cc：源文件，Unix/Linux

什么是多文件编程

又或者是有一天，你写出了很牛的功能代码，出于版权和保密考虑，大多是已经编译好的二进制文件，可能仅包含 .h 文件。

```
// 1.student.h
void printInfo();

// 2.student.cpp
#include "student.h"
//printInfo 定义

// 3.main.cpp
#include "student.h"
int main(){
    // ...
}
```

多文件编程详细步骤--头文件

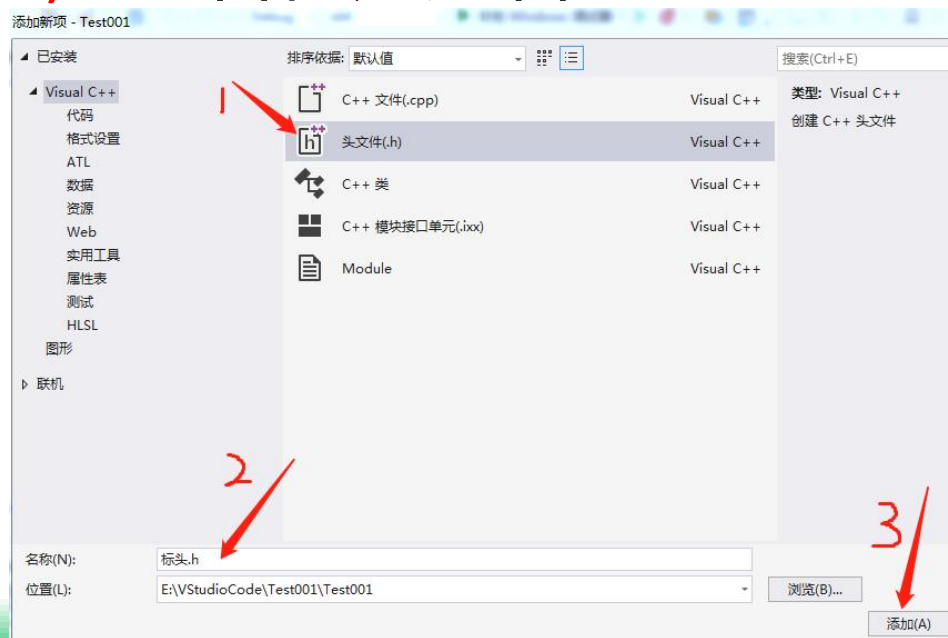
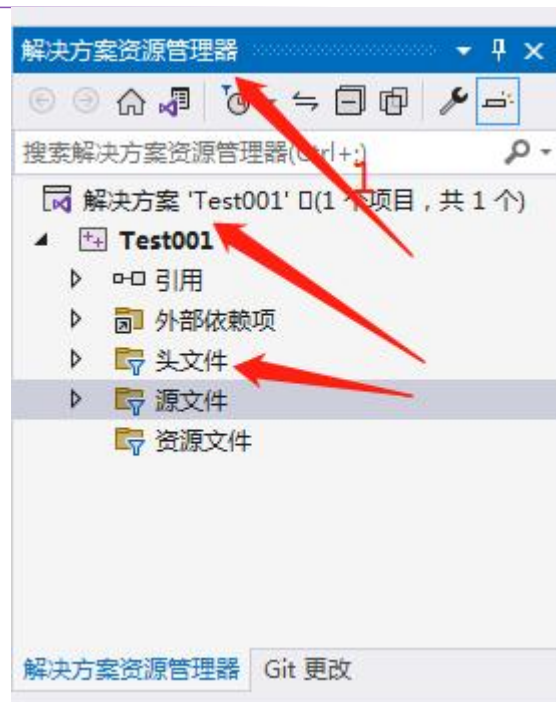
1.在资源管理器-解决方案中，找到**头文件**

2.在**头文件**文件夹上右键->添加->新建项

3.选择**头文件(.h)**->在名称处起名***.h

4.点击添加

5.编写代码



多文件编程详细步骤--源文件

- 1.在资源管理器-解决方案中，找到源文件
- 2.在源文件文件夹上右键->添加->新建项
- 3.选择C++文件 (.cpp)->在名称处起名***.cpp
- 4.点击添加
- 5.编写代码（注意：要包含***.h文件）



多文件编程详细步骤--main使用

`#include "***.h"` 即可使用

```
#include <iostream>
#include "student.h"
```

```
using namespace std;
```

```
int main() {
    printInfo();

    return 0;
}
```

```
#include <iostream>
using namespace std;
void printInfo();
```

```
#include "student.h"

void printInfo() {
    cout << "这是学生信息打印" << endl;
}
```

include <> 和 #include ""的区别

1 #include <>

#include <>引用的是编译器的**类库路径里面的头文件**。

一般用于引用标准头文件，如stdio.h，string.h等。

2 #include ""

#include ""引用的是项目相对路径中的头文件

一般用于引用自定义的头文件。

如果使用#include ""，首先会在当前项目所在文件夹下寻找是否有对应的头文件，**如果没有**，还是会到编译器自带头文件目录中查找。

例如，使用#include "stdio.h"，如果项目中不包含stdio.h这个头文件，则还是会定位到标准头文件中查找stdio.h这个文件。

防止头文件重复包含

- 1.使用宏定义避免重复引入
- 2.使用`#pragma once`避免重复引入
- 3.用`_Pragma`操作符

防止头文件重复包含

1.使用宏定义避免重复引入

```
#ifndef _NAME_H  
#define _NAME_H  
//头文件内容  
#endif
```

_NAME_H 是宏的名称。

注意:设置的宏名必须是**独一无二**的，不要和项目其他宏的名称相同。

当程序中第一次 #include 该文件时，由于 _NAME_H 尚未定义，所以会定义 _NAME_H 并执行“头文件内容”部分的代码；当发生多次 #include 时，因为前面已经定义了 _NAME_H，所以不会再重复执行“头文件内容”部分的代码。

防止头文件重复包含

2.使用#pragma once避免重复引入

`#pragma once`

`//放在文件开头位置`

`#ifndef` 是通过定义独一无二的宏来避免重复引入的，意味着每次引入头文件都要进行识别，所以效率不高。但考虑到 C 和 C++ 都支持宏定义，所以项目中使用 `#ifndef` 规避可能出现的“头文件重复引入”问题，不会影响项目的可移植性。

和 `#ifndef` 相比

`#pragma once` 不涉及宏定义，当编译器遇到它时就会立刻知道当前文件只引入一次，所以效率很高。

但值得一提的是，并不是每个版本的编译器都能识别 `#pragma once` 指令，老版本的编译器有些是不支持该指令（执行时会发出警告，但编译会继续进行），即 `#pragma once` 指令的兼容性不是很好。

除此之外，`#pragma once` 只能作用于某个具体的文件，而无法像 `#ifndef` 那样仅作用于指定的一段代码。

目前，几乎所有常见的编译器都支持 `#pragma once` 指令，甚至于 VS 2017 新建头文件时就会自带该指令。可以这么说，在 C/C++ 中，`#pragma once` 是一个非标准但却逐渐被很多编译器支持的指令。

防止头文件重复包含

3.用_Pragma操作符

`_Pragma("once")`

//放在文件开头位置

C99 标准中新增加了一个和 #pragma 指令类似的 _Pragma 操作符，其可以看做是 #pragma 的增强版，不仅可以实现 #pragma 所有的功能，更重要的是，_Pragma 还能和宏搭配使用。

防止头文件重复包含

考虑到编译效率和可移植性，`#pragma once` 和 `#ifndef` 经常被结合使用来避免头文件被重复引入

```
#pragma once
#ifndef _NAME_H
#define _NAME_H
//头文件内容
#endif
```

当编译器可以识别 `#pragma once` 时，则整个文件仅被编译一次！

如果编译器不识别 `#pragma once` 指令，此时仍有 `#ifndef` 在发挥作用！

课堂练习

利用多文件编程知识点，解决main里面调用student.h里面的多个方法。

课堂练习 代码

```
void func1();
```

```
void func2();
```

```
void func3();
```

```
...
```

//student.cpp具体实现，发挥自己的想象填充

知识点总结

- | | |
|---|----|
| 1. 为什么要多文件编程 | 了解 |
| 2. 多文件编程具体步骤 | 重点 |
| 3. <code>include <></code> 和 <code>#include ""</code> 的区别 | 熟悉 |
| 4. 防止头文件重复包含 | 重点 |

作业

说一说避免重复引用的方法有哪些？

下节课预告

二维数组